

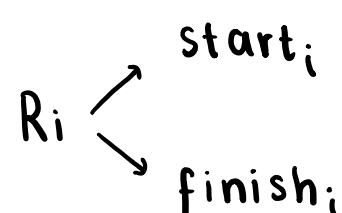
Lecture 17

08/10/24 10:02 AM

Interval Scheduling:

Requests $R_1, \dots, R_n$
Priority $w_1, \dots, w_n$

$$\text{OBJECTIVE: } \max_{S \subseteq \{R_1, \dots, R_n\}} \sum_{i \in S} w_i$$



* Case 1: R_n may belong to the optimal sequence.

$R_1 \quad R_2 \quad \dots \quad R_n$ (sorted in the order of finish times)

~ Remove all incompatible w/ R_n requests.

For a compatible job R_i , we must have $\text{finish}_i < \text{start}_n$

→ max. i such that $\text{finish}_i < \text{start}_n$

$p(j) = \max \{i \mid i < j, R_i \text{ is compatible with } R_j\} \Rightarrow$ finding the job with latest finish time that is compatible with R_j
 $p(n)$ may not be $\in [1, n-1]$ if no compatible jobs

$R_1, \dots, R_{p(n)}$ are compatible with R_n
but $R_{p(n)+1}, \dots, R_{n-1}$ are not.

Compute Optimal ($R_1, \dots, R_{p(n)}$)
 $\Rightarrow w_n + \text{Optimal} (R_1, \dots, R_{p(n)})$
could be $w_n + \{ \}$

* Case 2: R_n may not belong to optimal sequence

Compute Optimal ($R_1, \dots, R_{n-1}$)

$$\therefore \text{Optimal} (R_1, \dots, R_n) = \max \begin{cases} \text{Optimal} (R_1, \dots, R_{n-1}) \\ w_n + \text{Optimal} (R_1, \dots, R_{p(n)}) \\ w_n \end{cases}$$

$$\text{BASE CASE: } \text{Optimal} (R_i) = \max \begin{cases} 0 & \rightarrow \text{don't take } R_i \\ w_i & \rightarrow \text{take } R_i \end{cases}$$

W.L.O.G Assume that weights are positive

0-1 knapsack

Items $w_1, w_2, \dots, w_n \in \mathbb{Z}$
Values $v_1, v_2, \dots, v_n$

$$\max_{S \subseteq [n]} \sum_{i \in S} v_i \quad \text{subject to } \sum w_i \leq W$$

without this constraint, just choose all items to maximise value.

0-1 $\Rightarrow$ choose the entire item or don't choose item $\Rightarrow$ take it once or don't take it at all
fractional knapsack $\Rightarrow$ can choose fractions of items

Either $\text{Item}_n \in \text{Opt}$ or not.

* Case 1: $\text{Item}_n \in \text{Opt}$: $w_n \leq W$!!!
condition has to be checked
YES: $\text{Opt} ([1, n-1], W - w_n)$
NO: $\rightarrow$

* Case 2: $\text{Item}_n \notin \text{Opt}$: $\text{Opt} ([1, n-1], W)$

$$\text{Opt} ([1, n], W) = \max \begin{cases} \text{Opt} ([1, n-1], W - w_n) + v_n \\ \text{Opt} ([1, n-1], W) \end{cases}$$

No. of entries in memoization matrix = $n \times W$

$$\text{Opt} ([1, i], W')$$

at most 2^n possible values
n possible values

Say W is very very large.

$$W' = \sum_{i=1}^n a_i \cdot w_i \quad a_i \in \{0, 1\}^n$$

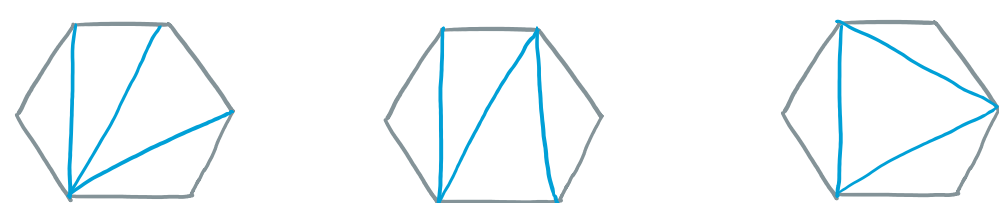
If W is small,

$$\sum_{i: a_i=1} w_i \leq W$$

$$\text{Opt} ([1, i], W - \sum_{j=i+1}^n \alpha_j \cdot w_j)$$

2^{n-i} possibilities
 $w_{i+1}, \dots, w_n$
 $\alpha_{i+1}, \dots, \alpha_n$
(n-i) items
 $\in \{0, 1\}$

Triangulation of Polygons



Cost = sum of lengths of diagonals introduced = S

We want to minimise cost.

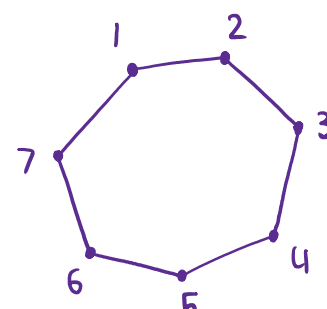
$$\Rightarrow \text{Minimise the sum of perimeters of the triangles} = 2S + \underline{C}$$

perimeter of polygon

* CONVEX POLYGON

For all points on the polygon,
the line joining them is inside the polygon.

$\Rightarrow \forall a_1, a_2 \in \text{polygon}$
 $\forall \lambda \in (0, 1)$
 $\lambda a_1 + (1-\lambda) a_2 \in \text{polygon}$

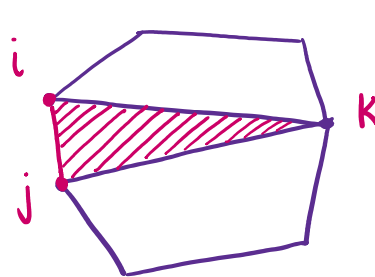


Cost of triangulation:

Let $A[i, j]$ denote the min. cost of triangulation of polygon with vertices $i, i+1, \dots, j-1, j$.

$$A[i, j] = \min_{i < k < j} \{ A[i, k] + A[k, j] + w_{i-k} + w_{k-j} + w_{j-i} \}$$

i to j is one edge



$j = i+1 \Rightarrow$ zero diagonals $\Rightarrow$ 0 cost of triangulation
 $j = i+2 \Rightarrow$ 1